

Google Java Style Guide

1 Introduction

This document serves as the **complete** definition of Google's coding standards for source code in the Java™ Programming Language. A Java source file is described as being *in Google Style* if and only if it adheres to the rules herein.

Like other programming style guides, the issues covered span not only aesthetic issues of formatting, but other types of conventions or coding standards as well. However, this document focuses primarily on the **hard-and-fast rules** that we follow universally, and avoids giving *advice* that isn't clearly enforceable (whether by human or tool).

1.1 Terminology notes

In this document, unless otherwise clarified:

1. The term *class* is used inclusively to mean a normal class, record class, enum class, interface or annotation type (`@interface`).
2. The term *member* (of a class) is used inclusively to mean a nested class, field, method, or *constructor*; that is, all top-level contents of a class except initializers.
3. The term *comment* always refers to *implementation* comments. We do not use the phrase "documentation comments", and instead use the common term "Javadoc."

Other "terminology notes" will appear occasionally throughout the document.

1.2 Guide notes

Example code in this document is **non-normative**. That is, while the examples are in Google Style, they may not illustrate the *only* stylish way to represent the code. Optional formatting choices made in examples should not be enforced as rules.

2 Source file basics

2.1 File name

For a source file containing classes, the file name consists of the case-sensitive name of the top-level class (of which there is [exactly one](#)), plus the `.java` extension.

2.2 File encoding: UTF-8

Source files are encoded in **UTF-8**.

2.3 Special characters

2.3.1 Whitespace characters

Aside from the line terminator sequence, the **ASCII horizontal space character (0x20)** is the only whitespace character that appears anywhere in a source file. This implies that:

1. All other whitespace characters are escaped in `char` and string literals and in text blocks.
2. Tab characters are **not** used for indentation.

2.3.2 Special escape sequences

For any character that has a [special escape sequence](#) (`\b`, `\t`, `\n`, `\f`, `\r`, `\s`, `\"`, `\'` and `\\`), that sequence is used rather than the corresponding octal (e.g. `\012`) or Unicode (e.g. `\u000a`) escape.

2.3.3 Non-ASCII characters

For the remaining non-ASCII characters, either the actual Unicode character (e.g. `∞`) or the equivalent Unicode escape (e.g. `\u221e`) is used. The choice depends only on which makes the code **easier to read and understand**, although Unicode escapes outside string literals and comments are strongly discouraged.

Tip: In the Unicode escape case, and occasionally even when actual Unicode characters are used, an explanatory comment can be very helpful.

Examples:

Example	Discussion
<code>String unitAbbrev = "μs";</code>	Best: perfectly clear even without a comment.
<code>String unitAbbrev = "\u03bcs"; //</code> <code>"μs"</code>	Allowed, but there's no reason to do this.
<code>String unitAbbrev = "\u03bcs"; //</code> <code>Greek letter mu, "s"</code>	Allowed, but awkward and prone to mistakes.
<code>String unitAbbrev = "\u03bcs";</code>	Poor: the reader has no idea what this is.
<code>return '\uffeff' + content; //</code> byte order mark	Good: use escapes for non-printable characters, and comment if necessary.

Tip: Never make your code less readable simply out of fear that some programs might not handle non-ASCII characters properly. If that should happen, those programs are **broken** and they must be **fixed**.

3 Source file structure

An ordinary source file consists of these sections, **in order**:

1. License or copyright information, if present
2. Package declaration
3. Imports
4. Exactly one top-level class declaration

Exactly one blank line separates each section that is present.

A `package-info.java` file is the same, but without the class declaration.

A `module-info.java` file does not contain a package declaration and replaces the class declaration with a module declaration, but otherwise follows the same structure.

3.1 License or copyright information, if present

If license or copyright information belongs in a file, it belongs here.

3.2 Package declaration

Every source file must have a package declaration. [Compact source files](#) are not used. (This rule obviously does not apply to `module-info.java` files, which have a different syntax that does not include a package declaration.)

The package declaration is **not line-wrapped**. The column limit (Section 4.4, [Column limit: 100](#)) does not apply to package declarations.

3.3 Imports

3.3.1 No wildcard imports

Wildcard ("on-demand") imports, static or otherwise, are not used.

3.3.1.1 No module imports

[Module imports](#) are not used.

Example:

```
import module java.base;
```

3.3.2 No line-wrapping

Imports are **not line-wrapped**. The column limit (Section 4.4, [Column limit: 100](#)) does not apply to imports.

3.3.3 Ordering and spacing

Imports are ordered as follows:

1. All static imports in a single group.
2. All non-static imports in a single group.

If there are both static and non-static imports, a single blank line separates the two groups. There are no other blank lines between imports.

Within each group the imported names appear in ASCII sort order. (**Note:** this is not the same as the import *lines* being in ASCII sort order, since `'.'` sorts before `';'`.)

3.3.4 No static import for classes

Static import is not used for static nested classes. They are imported with normal imports.

3.4 Class declaration

3.4.1 Exactly one top-level class declaration

Each top-level class resides in a source file of its own.